CS 6210 Spring 2026 Test 1

(120 min Canvas Quiz)

[Total Points: 109]

Table of Contents

OS Structure [36 points] 1
  Spin [9 points] 2
  L3 Microkernel [12 points] 2
  Exokernel/Spin/L3 [15 points] 3
Virtualization [21 points] 4
  Full vs Paravirtualization [8 points] 4
  Paravirtualization [4 points] 4
  Full virtualization [6 points] 5
  Memory Management [3 points] 5
Parallel Systems [32 points] 6
  Shared Memory Machines [4 points] 6
  Mutual Exclusion Locks [14 points] 6
  Lock/Barriers [6 points] 9
  LRPC and Scheduling [5 points] 10
  Tornado [3 points] 11
HIDDEN - Potpourri [20 points] 11

OS Structure [36 points]

Spin [9 points]

1. [5 points] A friend says that he recalls that there were two major strikes against SPIN in the way it handles accessing the endpoints of object interfaces (Create(), Combine(), Resolve()). He says this results in SPIN being not performant and also unsafe due to unprotected memory access between extensions on top of SPIN.

a) [4 points] With succinct bullets, explain how SPIN creates protection domains and use this to explain to your friend whether he was correct or not with regards to 1) the performance of SPIN OS and 2) the safety in regard to the isolation of the protection domains.

b) [1 point] While Modula-3 provides type safety for most SPIN extensions, certain extensions (like device drivers) must be designated as "trusted." What specific safety trade-off does SPIN make for these trusted extensions? Justify your answer.

2. [4 points] You have implemented MY-OS using the extensibility mechanisms in SPIN. There are two user level processes running on top of MY-OS. One is currently performing CPU intensive computational work that may last several hours. Another is running a webserver, with several non-CPU intensive requests coming in per minute.

How do you prevent the first process from hogging the CPU and stymieing the second process that is servicing http requests?

L3 Microkernel [12 points]

1. [2 points] Thomas, an indie game developer, is debugging a memory corruption issue. He wants to implement a custom debugger that uses specific page-fault handling logic (essentially acting as a custom memory pager) to track access.

With a few succinct bullets, explain why implementing this custom pager is structurally easier and safer in a microkernel architecture compared to a monolithic structure.

2. [10 points] You are the Lead Systems Architect for FlashTrade, an High Frequency Trading (HFT) firm. You are designing a specialized OS kernel on top of L3 microkernel to host four client trading algorithms on a single server while ensuring strict proprietary data isolation. The processor architecture you are targeting has the following features:

- A 32-bit hardware address space.

- Paged virtual memory system (8KB pages) with a processor register called PTBR that points to the page table in memory.
- A Tagged TLB supports tagging entries with Address Space IDs (ASIDs).
- A pair of hardware-enforced segment registers (base and limit) which restrict the virtual address range accessible by a process.
- A virtually indexed, physically tagged processor cache.

Your system runs a shared Kernel Lib (K), which requires 512 MB, and four client protection domains. Each client runs as a user level process. The clients use services provided by the Kernel Lib (libraries for network access, memory management, and CPU scheduling). You design the hardware address spaces for each client as follows:

- Client A: Kernel Lib (512 MB) + Trading Model (2.5 GB)
- Client B: Kernel Lib (512 MB) + Trading Model (2.5 GB)
- Client C: Kernel Lib (512 MB) + Trading Model (1.5 GB) + Forecast Model (1.5 GB)
- Client D: Kernel Lib (512 MB) + Trading Model (3 GB)

a) [2 points] Your friend asks you why you chose to map the Kernel Lib into each of the four client hardware address spaces. What is your justification?

b) [2 points] A client makes a system call contained in the Kernel Lib. How would this be handled in your design?

c) [2 points] Your friend is curious how your design gives the memory isolation guarantees for the clients. What is your answer?

d) [2 points] Answer True/False with justification. No credit without justification.

The design guarantees that the Kernel lib is protected from the client.

e) [2 points] Answer True/False with justification. No credit without justification.

The design guarantees that the clients are protected from the Kernel.

Exokernel/Spin/L3 [15 points]

1. [9 points] Based on the design principles outlined in the SPIN, Exokernel, and L3 papers, imagine that you are tasked with implementing a packet multiplexer. You want this to be fast since it sits on the critical path (examining every packet).

a) [3 points] Explain how you will implement this packet filter in (i) SPIN, (ii) Exokernel, and (iii) a microkernel.

b) [6 points] After you've implemented the code for the multiplexer, you realize that you introduced two undesirable bugs.

The first bug results in out-of-bound memory access that could potentially be very dangerous. Explain how this bug is dealt with in (i) SPIN (ii) Exokernel and (iii) a microkernel.

c) [6 points] The second bug is much more serious. You figured that in certain edge conditions, your packet multiplexer gets into an infinite loop. Explain how this bug would be dealt with in (i) SPIN (ii) Exokernel, and (iii) a microkernel.

Virtualization [21 points]

Full vs Paravirtualization [8 points]

1. [8 points]

a) [4 points] Your friend has analyzed the performance of a fully virtualized environment and XenoLinux on top of Xen. The workload she studied is one in which process creation happens very frequently. Her profile result shows that XenoLinux performs much better than the fully virtualized environment. Give succinct bullets to explain why this may be so.

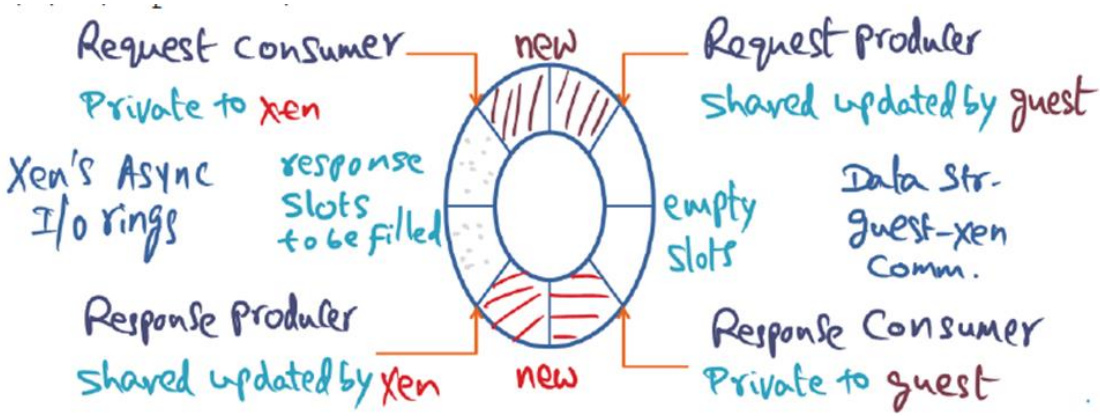
b) [4 points] Your friend also analyzed a workload consisting of a number of small memory footprint interactive processes running on top of a fully virtualized environment and XenoLinux on top of Xen. Will she see a significant difference in the performance of the two environments for this workload? Justify your answer.

Paravirtualization [4 points]

1. [2 points] Imagine a Linux Guest OS atop Xen (XenoLinux) that utilizes an asynchronous I/O ring data structure for device I/O. Suppose a multithreaded user application in XenoLinux issues ten independent blocking I/O system calls in parallel.

Assume the starting state for our measurement is that all ten requests have already been placed in the I/O ring by XenoLinux, and Xen is aware of them. If the primary metric to optimize is the number of context switches, how can XenoLinux optimize performance for receiving the responses, and exactly how many context switches will occur for XenoLinux to be notified that all ten requests are complete?

2. [2 points]



The image above shows the asynchronous I/O ring data structure used in Xen to facilitate communication between the guest OS and Xen. The request producer and response producer pointers are shared by Xen and the guest OS. Do either of these pointers require a mutual exclusion lock? Justify your answer.

Full virtualization [6 points]

1. [4 points]

Suppose you are a cloud provider using full virtualization to host multiple tenants on the same physical hardware.

a) [2 points] A malicious tenant gains root privileges within their own virtual machine and attempts to access the memory of another VM on the same host. Describe one mechanism by which full virtualization prevents this unauthorized memory access.

b) [2 points] A customer runs a specialized application that uses modern CPU instructions. The application runs correctly on bare metal but crashes when executed inside a fully virtualized VM in your cloud. What could be a possible reason for this failure?

2. [2 points] Answer True/False with justification. No credit without justification.

In VMware's full virtualization approach, the shadow page table maps the Guest Physical Page Number (PPN) to the Host Machine Page Number (MPN).

Memory Management [3 points]

1. [3 points] Suppose two virtual machines (VM-A and VM-B) are running on the same host. VMware ESX Server uses content-based page sharing, where a background process scans memory to find identical

pages by hashing their contents. Provide **two specific reasons** why the designers chose this transparent, hypervisor-driven approach over simply having the guest OS explicitly notify the hypervisor when pages can be shared.

Parallel Systems [32 points]

Shared Memory Machines [4 points]

1. [4 points]

Consider the following execution on a shared memory multiprocessor. T1, T2, and T3 are executing concurrently on different processors.

Assume all the shared variables are initially 0.

Thread T1	Thread T2	Thread T3
I1: a = a + 1 I2: b = b + 1	I3: c = a + b	I4: d = c I5: e = a

Assume that the statements in the program compile to instructions which execute atomically, are not re-ordered by the compiler, and appear to the interconnection network in program-order. The multiprocessor implements Lamport's sequential consistency memory model.

Which of the following final values is/are **impossible** with the above execution? (+2 for correct choice; -1 for incorrect choice)

(i) a = 1; b = 1; c = 2; d = 2; e = 0
(ii) a = 1; b = 1; c = 0; d = 0; e = 0
(iii) a = 1; b = 1; c = 1; d = 0; e = 0
(iv) a = 1; b = 1; c = 1; d = 2; e = 1
(v) a = 1; b = 1; c = 1; d = 1; e = 1
(vi) a = 1; b = 1; c = 2; d = 0; e = 0

Mutual Exclusion Locks [14 points]

1. [8 points]

Given:

- 32-core cache-coherent bus-based multiprocessor
- Invalidation-based cache coherence protocol
- Architecture supports atomic "Test-and-set (T&S)", atomic "Fetch-and-add (F&inc)", and atomic "fetch-and-store (F&St)" operations. All these operations bypass the cache.
- An application has 32 threads, one on each core.
- ALL threads are contending for the SAME lock (L)

- Each lock acquisition results in 100 iterations of the spin loop for each thread

The questions are with respect to the following spin-lock algorithms (as described in the MCS paper, and restated below for convenience):

- Spin on Test-and-Set: The algorithm performs a globally atomic T&S on the lock variable "L"
- Spin on Read: The algorithm, on failure to acquire the lock using T&S, spins on the cached copy of "L" until notified through the cache coherence protocol that the current user has released the lock.
- Ticket Lock: The algorithm performs "fetch_and_add" on a variable "next_ticket" to get a ticket "my_ticket". The algorithm spins until "my_ticket" equals "now_serving". Upon lock release, "now_serving" is incremented to let the spinning threads that the lock is now available.
- MCS lock: The algorithm allocates a new queue node, links it to the head node of Lock queue using "fetch-and-store", sets the "next" pointer of the previous lock requestor to point to the new queue node, and spins on a "got_it" variable inside the new queue node if the lock is not immediately available (i.e., the Lock queue is non-empty). Upon lock release, using the "next" pointer, the next user of the lock is notified that they have the lock.

a) [2 points] How many bus accesses are incurred per lock acquisition in the "Spin on T&S" algorithm? No credit without justification.

b) [2 points] This pertains to the "Spin on Read" algorithm. One thread is in the critical section governed by the lock. All the other threads are spinning waiting their turns. How many T&S operations happen upon lock release? No credit without justification.

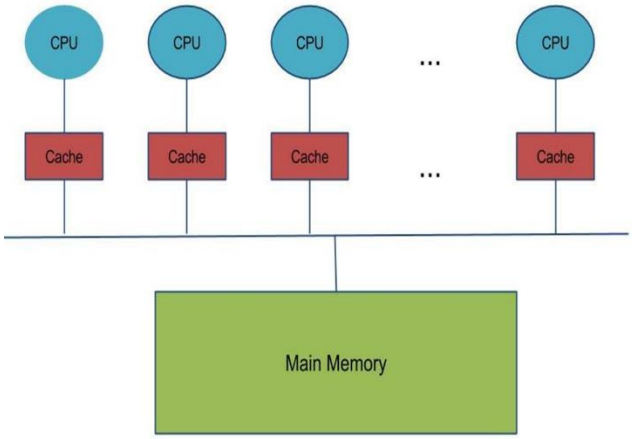
c) [2 points] This pertains to the "Ticket Lock" algorithm. One thread is in the critical section governed by the lock. All the other threads are spinning waiting their turns. How many cache reload operations happen upon lock release? No credit without justification.

d) [2 points] This pertains to the "MCS lock" algorithm. Answer True/False with justification. No credit without justification.

At lock release, if the "next" pointer is "nil" it is safe for the MCS lock algorithm to assume that there are no other threads waiting for this lock.

2. [6 points]

You have designed a bus-based custom non-cache-coherent shared memory DSP (Digital Signal Processor). Each CPU in the DSP has a private cache.



The hardware provides the following primitives for the interaction between the private cache of a CPU and the shared memory:

1. fetch(addr): Pulls the latest value from main memory into the cache
2. flush(addr): Pushes the value at addr in the cache to main memory; it does not evict it from the cache
3. hold(addr): Locks the memory bus for addr; no other core can fetch or flush this address until released
4. unhold(addr): Releases the lock on addr

You got this generic implementation for a ticket lock algorithm and tried it on your architecture. It did not work.

```
struct ticket_lock {
    int next_ticket;  // The next ticket number to give out    int now_serving;  // The ticket number currently allowed to enter
};

void lock(struct ticket_lock *l) {
    // Acquire ticket
    int my_ticket = l->next_ticket++;

    // Wait for turn
    while (l->now_serving != my_ticket) {
        // Spin
```

```
    }
}

void unlock(struct ticket_lock *l) {
    l->now_serving++;   // Release
}
```

a) [1 point] Identify any one potential flaw in the lock function when implemented on your architecture.

b) [1 point] Identify any one potential flaw in the unlock function when implemented on your architecture.

c) [4 points] We have provided a skeleton of the fixed lock() and unlock() functions below. Fill in the numbered blanks (1 through 4) with the correct primitive (fetch, flush, hold, or unhold) to make the ticket lock function correctly on your non-cache-coherent DSP.

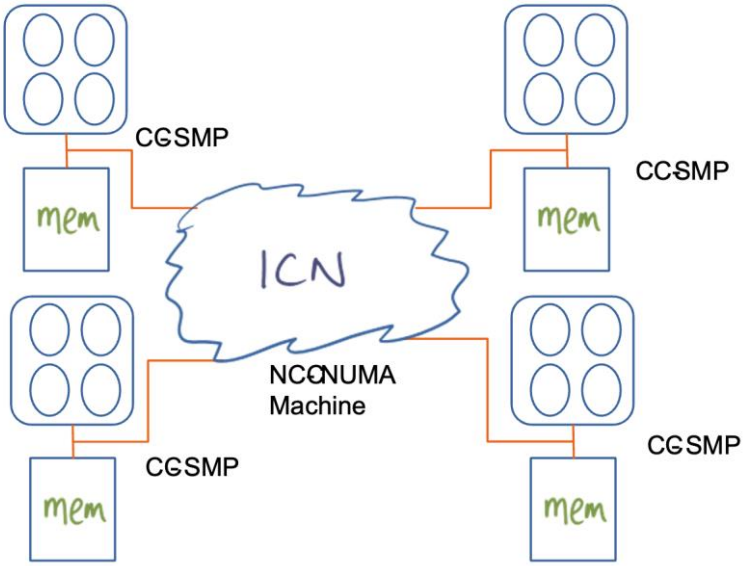
```
void lock(struct ticket_lock *l)
{
      // Acquire ticket atomically
      __1__(&l->next_ticket);
      fetch(&l->next_ticket);
      int my_ticket = l->next_ticket++;
      __2__(&l->next_ticket);
      unhold(&l->next_ticket);

      // Wait for turn
      fetch(&l->now_serving);
      while (l->now_serving != my_ticket) {
            ____3____(&l->now_serving);
      }
}

void unlock(struct ticket_lock *l)
{
      // Release
      l->now_serving++;
      __4__(&l->now_serving);
}
```

Lock/Barriers [6 points]

1. [6 points] Consider an NCC NUMA machine. Each node in the NUMA machine is CC-SMP. How will you implement MCS barrier on such a machine?



LRPC and Scheduling [5 points]

1. [5 points]

LRPC's performance improvement relies on separating the setup costs from the actual call cost. The "Binding phase" is used to setup the communication channel for future calls. During this phase, the kernel allocates a shared argument stack (A-stack) mapped into both the client and server address spaces. A binding object is then created for authorization.a) [2 points] Even though the shared memory channel has been configured, the client still requires a kernel trap to invoke the server procedure. Give any two valid reasons why this trap is required.

b) [1 point] A server expects a pointer to a linked list as a formal parameter. You are writing the client-stub and server-stub to implement the client-server interaction in LRPC. Initially, you placed the pointer to a linked list in the client process's execution stack as the actual parameter in the A-stack, which can be retrieved by the server-stub during the actual call and placed on the server's E-Stack.

Why will this not work?

c) [2 points] A server expects a pointer to a linked list as a formal parameter. You are writing the client-stub and server-stub to implement the client-server interaction in LRPC. Initially, you placed the pointer to a linked list in the client process's execution stack as the actual parameter in the A-stack, which can be retrieved by the server-stub during the actual call and placed on the server's E-Stack.

How can you fix it in writing the client and server stubs?

Tornado [3 points]

1. [3 points] The paper shows that using multiple representations (reps) for the Process object improves page fault handling performance significantly but worsens the performance for region deletion as shown in the graphs below:

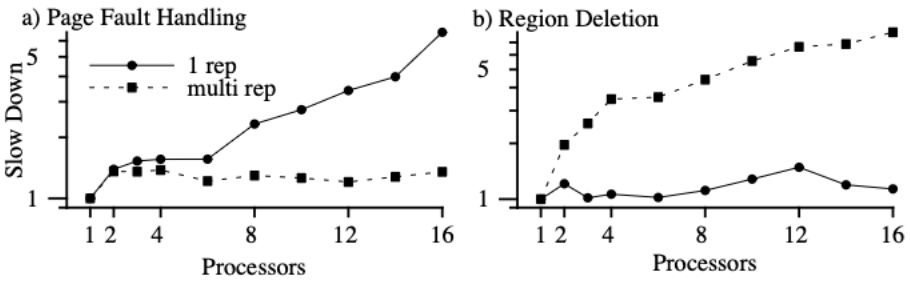


Figure 5: *Comparison of the performance of a Process clustered object with one rep vs. a Process clustered object with n reps (one per processor) for (a) in-core page fault handling and (b) Region destruction*

a) [2 points] Given this figure, give two reasons why replicating the Process object is still a good idea.

b) [1 point] Explain why region destruction takes more time.

HIDDEN - Potpourri [20 points]

1. [4 points]

a) [2 points] Identify a pro and a con for the fixed-processor scheduling policy.

Ans:

- (+1) Pro: At heavy loads the cache will likely be polluted. Fixed-processor policy is quick
- (+1) Con: There could be a long latency before a thread is scheduled if the queue on that processor happens to be long

b) [2 points] Identify a pro and con for the minimum intervening (MI) threads scheduling policy

Ans:

- (+1) Pro: At light/moderate loads, increases the chance that the working set of a thread will be in the nearer levels of the processor cache
- (+1) Con: (either acceptable)
  - The data structures needed in the scheduler for MI is quite elaborate (affinity for each thread with respect to ALL the processors). Thus, the scheduler may take quite some time to assign a thread to a processor.
  - The processor chosen for a thread may already have a long queue of waiting threads to be scheduled on it so the choice may not be a good one by the time the thread runs on that processor

2. [2 points] (Answer True/False with justification; no credit without justification)

In VM oblivious page sharing, if the hypervisor identifies that a newly scanned guest page has a hash value identical to an entry already present in its global hash table, it can immediately reclaim the physical memory of the new page and point the guest to the existing machine page.

Ans: [+2] False. The hash match is only a hint, not an absolute. Due to the possibility of hash collisions, the hypervisor must perform a full byte-by-byte comparison of the two machine pages to confirm they are truly identical before the memory can be safely shared, and the redundant page discarded.

3. [2 points] Dissemination barrier algorithm can be implemented on either a shared memory multiprocessor or a message-passing cluster. Suppose you deploy this algorithm at a large scale (100,000 nodes) on a message-passing cluster, where nodes communicate strictly via explicit network messages, and the interconnect is shared among all nodes. How will the efficacy of the barrier in this environment compare to a shared memory multiprocessor? Will it perform better, worse, or show no difference? Justify your answer.

Ans:

- [+1] Worse.
- [+1] Justification: Network congestion. Each round injects O(P) messages concurrently into the shared interconnect leading to contention/queuing delay increase (and there are O(log P) rounds, so delays accumulate).

4. [2 points] In simple terms, give your understanding of the concept of a “clustered object” in the Tornado OS.

Ans: Same object reference (i.e., logical) may de-reference to different physical representations in memory.

5. [2 points]

Answer True or False with justification. No credit without justification.

An MCS lock has lower latency than a simple test-and-set lock when there is no contention.

Ans:

- [+2] False. T&S would require a single atomic operation. An uncontended MCS lock requires a fetch-and-store *plus* the overhead of allocating/setting up the queue node, resulting in slightly higher latency.
- [+0] No credit for True, or False w/o valid justification

6. [2 points] Answer True or False with justification. No credit without justification.

In a k-ary tree barrier, increasing k decreases both the number of internal nodes and the number of threads that may simultaneously spin on each internal node.

Ans:

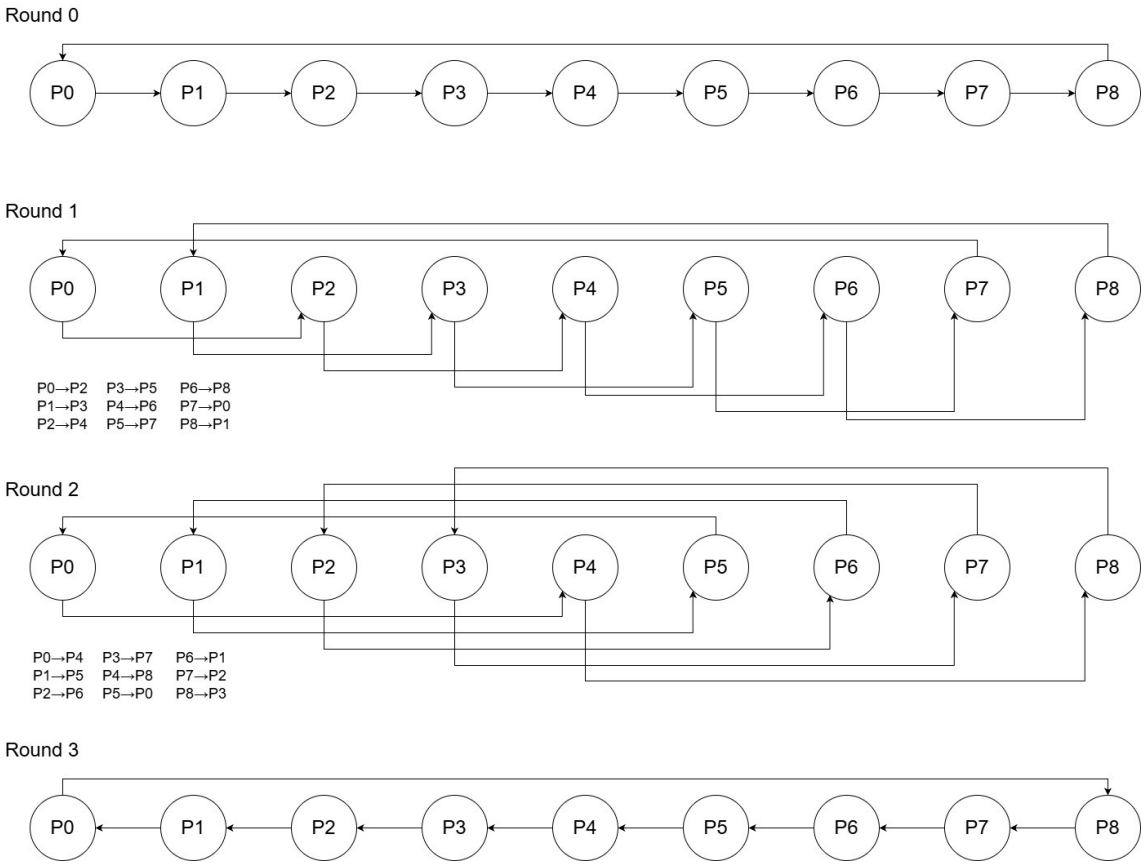
- [+2] False. Larger k results in fewer internal nodes but consequently increases the number of children at each node, meaning more threads will spin on a single parent node.
- [+0] No credit for True, or False w/o valid justification

7. [2 points] Answer Yes/No with justification/correct answer. (No credit without justification).

An exokernel must be able to take resources back from a library OS that fails to respond to revocation requests. Does the exokernel simply kill the offending library OS process to reclaim the resource?

Ans: [+2] No. The exokernel uses an "abort protocol" where it forcibly reclaims the specific resource (e.g., breaking the secure bindings/hardware mappings) and informs the library OS.

8. [4 points] The following picture shows a dissemination barrier on processors P0…P8 over Rounds 0–3. Each directed edge indicates which processor sends ”arrived” message to which other processor during that round.



Focusing on processor P6, what does P6 know at the end of each round (0, 1, 2, 3) as to which processors have arrived at the barrier? Justify your answer.

Ans:

- [+1] P6 knows {P5, P6}.
- [+1] P6 knows {P3, P4, P5, P6}.
- [+1] P6 knows {P0, P1, P2, P3, P4, P5, P6, P8} (missing only P7).
- [+1] P6 knows {P0, P1, P2, P3, P4, P5, P6, P7, P8} (all).